

Resume Insight Analyzer - Upgrade Summary

Technical Documentation of the 10 Core Application Enhancements

Change 1: Job Description Input & Core Matching

- What changed: Added a Job Description input text area alongside the resume text area, extracting required skills from both texts and calculating exact matched and missing skill sets.
- Files modified: ResumeRequest.java, ResumeResponse.java, ResumeAnalyzerService.java, index.html, script.js
- Why this change: Allows users to analyze their resumes against specific target job descriptions instead of generic roles, offering a personalized match score.
- Interview summary: I extended the matching engine to extract and compare skills from user-provided job descriptions, offering tailored skill alignment reports alongside general role suggestions.

Change 2: PDF Resume Upload via Apache PDFBox

- What changed: Integrated Apache PDFBox PDFTextStripper to parse uploaded PDF resumes, extract the raw text content, and feed it directly into the analysis text area.
- Files modified: pom.xml, ResumeController.java, index.html, script.js
- Why this change: Pasting resumes manually is a tedious user experience. Allowing file uploads increases usability and modernizes the application workflow.
- Interview summary: I integrated Apache PDFBox to enable direct PDF uploads, extracting text programmatically on the backend and populating the analyzer.

Change 3: Regex Word-Boundary Skill Matching & Synonyms

- What changed: Refactored keyword detection using regex boundaries (\b) to avoid token split errors on hyphenated skills (like scikit-learn) and added synonym mapping (e.g., JS to JavaScript).
- Files modified: ResumeAnalyzerService.java
- Why this change: Fixed a critical bug where compound skills split on non-letters could never match, and normalized common variations of technology terms.
- Interview summary: I optimized the matching engine using regex word boundaries to accurately capture compound skills like scikit-learn and resolved synonyms using a mapping dictionary.

Change 4: Spelling & Grammar Check via LanguageTool

- What changed: Embedded the JLanguageTool library to run spelling and grammar analysis on the resume text, returning a structured list of warnings and correction recommendations.
- Files modified: pom.xml, GrammarIssue.java, ResumeAnalyzerService.java, index.html, script.js
- Why this change: Typos on resumes lead to automatic candidate rejection; adding automated grammar scanning helps users find mistakes before submitting.
- Interview summary: I embedded the open-source LanguageTool library to automatically flag spelling and grammar mistakes in resumes, complete with correction suggestions.

Change 5: JPA Persistence & PostgreSQL Logging

- What changed: Configured Spring Data JPA with PostgreSQL to save the results of every analysis (filename, text, score, matched/missing skills, grammar issues) and built a historical logs feed.
- Files modified: pom.xml, AnalysisHistory.java, AnalysisHistoryRepository.java, ResumeController.java, index.html, script.js
- Why this change: Saves past analysis runs so that users can view their application logs, reload previous configurations, and review roadmaps.
- Interview summary: I added persistence using Spring Data JPA and PostgreSQL to record historical analyses, allowing users to reload previous configurations.

Change 6: Opt-in AI Learning Roadmap (Gemini)

- What changed: Configured RestTemplate to call the Google Gemini API (gemini-3.1-flash-lite) using the missing skills as a prompt, generating an opt-in 3-month timeline study plan.
- Files modified: ResumeController.java, index.html, script.js
- Why this change: Helps candidates bridge their technical skills gaps by suggesting structured resources, timelines, and project topics.
- Interview summary: I implemented an opt-in AI roadmap feature using Google's Gemini API that generates a structured 3-month study timeline to bridge detected skill gaps.

Change 7: Multi-page PDF Report Generation

- What changed: Developed custom PDF export code using PDFBox to write wrapped, multi-page reports containing match score, skills clouds, grammar alerts, and the AI roadmap.
- Files modified: PDFGeneratorUtil.java, ResumeController.java, index.html, script.js
- Why this change: Enables users to download, store, and print high-quality analysis reports for offline reading and presentation.
- Interview summary: I authored a PDF generation utility using PDFBox that exports a well-formatted multi-page report containing the analysis dashboard results.

Change 8: Chart.js Dashboard Visualization

- What changed: Integrated Chart.js CDN on the frontend, rendering a horizontal bar chart that visually represents the match score index.
- Files modified: index.html, script.js
- Why this change: Replaces numeric strings with interactive visual representations, making match scores instantly readable and professional.
- Interview summary: I integrated Chart.js to present resume-matching indices as interactive horizontal bar charts, enhancing visual data analysis.

Change 9: Cloud-Ready Env Configurations

- What changed: Bound database credentials to environment variables, added H2 configurations for isolated maven tests, and programmatically parsed Render's DATABASE_URL string.
- Files modified: application.properties, DatabaseConfig.java, pom.xml
- Why this change: Decoupled credentials from source code for safety and resolved PostgreSQL connection failures during test runs and Render cloud builds.
- Interview summary: I modernized the application's configuration by binding credentials to env vars and adding a parser to cleanly connect to Render's native PostgreSQL strings.

Change 10: Tailored Design & Space Scale

- What changed: Redesigned frontend styling with Space Grotesk (headers), Inter (body), a teal/amber (#0D7377 / #E8A33D) palette, 8px spacing blocks, and sharp borders.
- Files modified: index.html, style.css, script.js
- Why this change: Transforms the basic static look into a high-end dashboard interface, establishing visual hierarchy and scannability.
- Interview summary: I redesigned the frontend using Space Grotesk typography, a teal/amber palette, and an 8px spacing system, achieving a premium responsive dashboard.